Лабораторная работа №2

Выполнил: Казаков А.Е.

Группа: 4404

Задание №1. Напишите программу перемножения больших матриц.

Сравните врем выполнения последовательной и параллельной программы на 2,4, 8 потоках (процессорах)

Краткое описание решения.

Программа реализована на языке C# (.NET 8). Умножение матриц выполняется классическим тройным циклом i-j-k (без использования библиотечных функций BLAS). Последовательная версия — обычный тройной вложенный цикл. Параллельная версия — внешний цикл по i распараллелен с помощью Parallel.For с параметром MaxDegreeOfParallelism = threads (аналог директивы #pragma omp parallel for в OpenMP). Распараллеливание внешнего цикла корректно, поскольку каждая итерация записывает в собственную строку результирующей матрицы — зависимости по данным между итерациями нет. Число потоков и размер матрицы передаются через аргументы командной строки. Время измеряется через Stopwatch. Корректность проверяется по контрольной сумме элементов результирующей матрицы — она должна быть одинаковой во всех запусках.

Решение.

Исходный код программы в двух частях:

Часть 1. Подключение пространств имён, объявление класса Program, метод Main: разбор аргументов командной строки, вывод размера матрицы и числа потоков, инициализация матриц a и b, запуск таймера Stopwatch и ветвление между последовательной и параллельной версиями умножения.

```
using System;
using System.Diagnostics;
using System.Threading.Tasks;

Ссылок: 0
class Program
{
    Ссылок: 0
    static void Main(string[] args)
    {
        if (args.Length < 2)
        {
            Console.WriteLine("Использование: dotnet run <размер_матрицы> <число_потоков>");
            Console.WriteLine("  число_потоков = 0 -> последовательная версия");
            return;
        }

        int n = int.Parse(args[0]);
        int threads = int.Parse(args[1]);

        Console.WriteLine($"Размер матрицы: {n}x{n}");
        Console.WriteLine($"Потоков: {(threads == 0 ? "последовательно" : threads.ToString())}");

        // Инициализация матриц
        double[,] a = new double[n, n];
        double[,] b = new double[n, n];
        double[,] c = new double[n, n];

        var rnd = new Random(42);
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
            {
                a[i, j] = rnd.NextDouble();
                b[i, j] = rnd.NextDouble();
            }

        var sw = Stopwatch.StartNew();

        if (threads == 0)
        {
            // Последовательная версия
            MultiplySequential(a, b, c, n);
        }
        else
        {
            // Параллельная версия
            MultiplyParallel(a, b, c, n, threads);
```

Часть 2. Функция MultiplySequential с классическим тройным вложенным циклом i-j-k и функция MultiplyParallel, в которой внешний цикл по i распараллелен через Parallel.For с параметром MaxDegreeOfParallelism, равным числу потоков.

```
            MultiplyParallel(a, b, c, n, threads);
        }

        sw.Stop();

        Console.WriteLine($"Время выполнения: {sw.ElapsedMilliseconds} мс");

        // Небольшая проверка корректности (сумма элементов)
        double sum = 0;
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                sum += c[i, j];
        Console.WriteLine($"Контрольная сумма: {sum:F4}");
    }

    Ссылок: 1
    static void MultiplySequential(double[,] a, double[,] b, double[,] c, int n)
    {
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                double s = 0;
                for (int k = 0; k < n; k++)
                    s += a[i, k] * b[k, j];
                c[i, j] = s;
            }
        }
    }

    Ссылок: 1
    static void MultiplyParallel(double[,] a, double[,] b, double[,] c, int n, int threads)
    {
        var options = new ParallelOptions { MaxDegreeOfParallelism = threads };

        // Распараллеливаем внешний цикл по i — аналог #pragma omp parallel for
        Parallel.For(0, n, options, i =>
        {
            for (int j = 0; j < n; j++)
            {
                double s = 0;
                for (int k = 0; k < n; k++)
                    s += a[i, k] * b[k, j];
                c[i, j] = s;
            }
        });
    }
```

Результаты запуска программы.

Запуск программы: dotnet run -c Release -- 500 0. Размер матрицы 500×500, последовательное выполнение. Время выполнения: 259 мс. Контрольная сумма: 31233871,1499.

```
PS C:\Users\0\Desktop\3123123123\lab2> dotnet run -c Release -- 500 0
Размер матрицы: 500x500
Потоков: последовательно
Время выполнения: 259 мс
Контрольная сумма: 31233871,1499
```

Запуск программы: dotnet run -c Release -- 500 2. Размер матрицы 500×500, 2 потока. Время выполнения: 218 мс. Контрольная сумма совпадает с последовательной версией — 31233871,1499, что подтверждает корректность распараллеливания.

```
PS C:\Users\0\Desktop\3123123123\lab2> dotnet run -c Release -- 500 2
Размер матрицы: 500x500
Потоков: 2
Время выполнения: 218 мс
Контрольная сумма: 31233871,1499
```

Запуск программы: dotnet run -c Release -- 500 4. Размер матрицы 500×500, 4 потока. Время выполнения: 83 мс. Контрольная сумма: 31233871,1499.

```
PS C:\Users\0\Desktop\3123123123\lab2> dotnet run -c Release -- 500 4
Размер матрицы: 500x500
Потоков: 4
Время выполнения: 83 мс
Контрольная сумма: 31233871,1499
```

Запуск программы: dotnet run -c Release -- 500 8. Размер матрицы 500×500, 8 потоков. Время выполнения: 65 мс. Контрольная сумма: 31233871,1499.

```
PS C:\Users\0\Desktop\3123123123\lab2> dotnet run -c Release -- 500 8
Размер матрицы: 500x500
Потоков: 8
Время выполнения: 65 мс
Контрольная сумма: 31233871,1499
```

Сводная таблица результатов.

Режим	**Время, мс**	**Ускорение $S = T_1 / T_n$**
Последовательно (1 поток)	259	1,00×
2 потока	218	1,19×
4 потока	83	3,12×
8 потоков	65	3,98×

Вывод. В ходе работы я написал программу для перемножения матриц на языке C# и распараллелил её с помощью Parallel.For. Последовательная и параллельная версии дают одинаковую контрольную сумму, значит, программа считает правильно. При размере матрицы 500 на 500 на восьми потоках время сократилось с 259 до 65 миллисекунд, то есть почти в четыре раза. Полного ускорения в восемь раз не получилось, потому что часть

времени уходит на создание потоков и синхронизацию, а сама задача умножения матриц сильно зависит от скорости работы с памятью. Особенно это заметно при переходе с одного потока на два: время почти не изменилось, потому что оба потока читают данные из одной и той же оперативной памяти. Лучший прирост получился при переходе с двух потоков на четыре, а дальше увеличение числа потоков даёт уже меньший эффект.

Ответы на вопросы

Вопрос 1. Почему распараллеливание внешнего цикла по i корректно?

Потому что каждая итерация внешнего цикла записывает результат в собственную строку результирующей матрицы c[i, *], которая не пересекается со строками, обрабатываемыми другими итерациями. Зависимости по данным между итерациями нет, поэтому распараллеливание безопасно.

Вопрос 2. Какие зависимости по данным могли бы помешать распараллеливанию?

Например, если бы распараллеливался внутренний цикл по k (в той же схеме) — тогда все итерации писали бы в одну и ту же ячейку c[i, j], что привело бы к гонке данных. Также опасно распараллеливание цикла, где одна итерация читает данные, которые записаны другой итерацией.

Вопрос 3. Почему ускорение не идеально линейное?

Причины:

1) Накладные расходы на создание/синхронизацию потоков и работу пула задач .NET.

2) Закон Амдала — доля последовательного кода в программе (инициализация матриц, суммирование контрольной суммы) ограничивает ускорение.

3) Пропускная способность памяти — при умножении матриц процессоры быстро упираются в шину памяти, а не в вычислительную мощность. Особенно это видно при переходе с 1 на 2 потока (259 → 218 мс), когда два ядра «делят» один и тот же поток данных из RAM.

5) Число физических ядер — если в системе меньше 8 физических ядер (например, 4 ядра + Hyper-Threading), 8 потоков работают на 4 ядрах и не дают линейного ускорения.

Вопрос 4. Что происходит при числе потоков больше, чем физических ядер?

Потоки начинают конкурировать за одни и те же ядра. Операционная система переключает их с потерей производительности на переключение контекста. Ускорение перестаёт расти или даже падает.